



Nombre y Apellido:

Legajo:

Examen Parcial 1

1. A continuación se presenta la sintaxis de un lenguaje imperativo simple, sin bucles y una semántica *small-step* para los comandos.

Sintaxis abstracta:

$$e ::= n \mid x \mid e + e$$

$$c ::= x := e \mid c; c \mid \text{ifz } e \ c \ c \mid \text{skip}$$

donde $n \in \mathbb{N}$ y $x \in \text{Var}$.

Semántica:

La semántica operacional está dada por una relación *big-step* $\Downarrow \subseteq (e \times \Sigma) \times \mathbb{N}$ para las expresiones del lenguaje, y una relación *small-step* $\rightarrow \subseteq (c \times \Sigma) \times (c \times \Sigma)$ para los comandos. Para las expresiones asumimos una semántica usual, mientras que para los comandos la semántica está definida por las siguientes reglas:

$$\frac{\langle e, \sigma \rangle \Downarrow n}{\langle x := e, \sigma \rangle \rightarrow \langle \text{skip}, \sigma[x \mapsto n] \rangle} \quad (\text{ASSG}) \qquad \frac{\langle e, \sigma \rangle \Downarrow n \quad n \neq 0}{\langle \text{ifz } e \ c_1 \ c_2, \sigma \rangle \rightarrow \langle c_1, \sigma \rangle} \quad (\text{IFZ-1})$$

$$\langle \text{skip}; c, \sigma \rangle \rightarrow \langle c, \sigma \rangle \quad (\text{SEQ-1})$$

$$\frac{\langle c_1, \sigma \rangle \rightarrow \langle c, \sigma_1 \rangle}{\langle c_1; c_2, \sigma \rangle \rightarrow \langle c; c_2, \sigma_1 \rangle} \quad (\text{SEQ-2}) \qquad \frac{\langle e, \sigma \rangle \Downarrow 0}{\langle \text{ifz } e \ c_1 \ c_2, \sigma \rangle \rightarrow \langle c_2, \sigma \rangle} \quad (\text{IFZ-2})$$

- a) Enunciar la propiedad de determinismo para la relación $\rightarrow$. Probar que la misma es determinista por inducción en la derivación, suponiendo que $\Downarrow$ es determinista.
- b) Se desea extender el lenguaje con un constructor que permita la ejecución paralela de comandos. Para ello se extenderá la sintaxis con un nuevo constructor:

$$c ::= \dots \mid c \text{ par } c$$

El comando $c_1 \text{ par } c_2$ ejecutará los comandos c_1 y c_2 pero de manera intercalada, pudiendo arrojar diferentes resultados. Por ejemplo, la ejecución de:

$$x := 1 \text{ par } (x := 2; x := x + 2)$$

puede arrojar 3 resultados diferentes para x : 4, 1 y 3, correspondientes a las ejecuciones de: $x := 1; x := 2; x := x + 2$, $x := 2; x := x + 2; x := 1$ y $x := 2; x := 1; x := x + 2$ respectivamente.

Agregar reglas semánticas para contemplar este nuevo constructor.

- c) Para permitir otra forma de paralelismo donde los códigos se ejecutan como una operación atómica, pero en cualquier orden se extenderá el lenguaje con otro constructor:

$$c ::= \dots \mid c \text{ parAtom } c$$

Por ejemplo, la ejecución de $x := 1 \text{ parAtom } x := 2; x := x + 2$ podrá arrojar 2 resultados posibles para x : 1 o 4.

Extender las reglas semánticas para contemplar este nuevo constructor.

- d) A continuación se define la sintaxis concreta para los comandos del lenguaje sin las extensiones:

$$c ::= x := e \mid c'; c \mid \text{'ifz' } e \ c \ c' \text{'end'} \mid \text{'skip'}$$

Para representar los constructores se definieron los siguientes tipos de datos:

```
data Exp = Var String | Num Int | Add Exp Exp
data Com = Seq Com Com | Assg String Exp | Ifz Exp Com Com | Skip
```

Suponiendo que se cuenta con la definición de un parser de expresiones del lenguaje (`parseExp :: Parser Exp`), completar la definición del parser de comandos, dando una definición de la función `parseCom`.

```
parseCom :: Parser Com
parseCom = ...
parseL :: Parser [Com]
parseL = sepBy1 parseCom (symbol ";")
parseCode :: Parser Com
parseCode = do xs ← parseL
              if null xs then failure
              else return (foldr Seq (last xs) (init xs))
```

2. Usando la codificación de naturales (como numerales de Church), pares y listas dadas en clase, definir las siguientes funciones en λ -cálculo, sin utilizar operadores de punto fijo. Puede usar los constructores de tipos: `zero`, `suc`, `pair`, `nil` y `cons` y las funciones `suma`, `fst` y `snd` sin definir.

- Definir una función `exp` que dado un par de naturales (x, y) calcule la potencia del 1º número del par con el segundo, es decir x^y .
- Definir una función `sumaPotencias` que dada una lista de pares de naturales $[(x_1, y_1), \dots, (x_n, y_n)]$ calcule la suma de las potencias de los pares, es decir $x_1^{y_1} + \dots + x_n^{y_n}$.